
AI時代のソフトウェア(*)開発

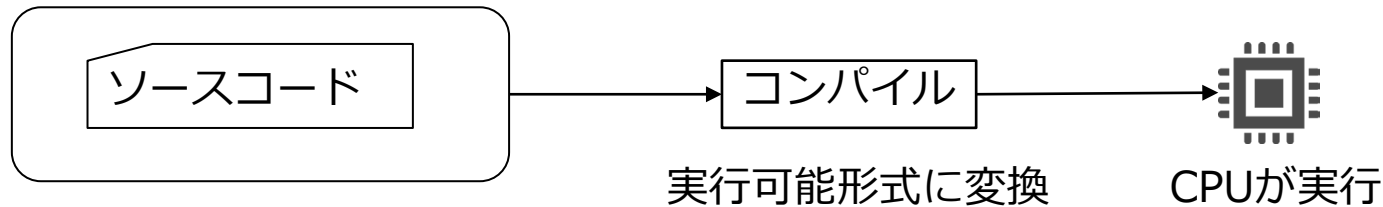
(*)ソフトウェア （引用：<https://ja.wikipedia.org/wiki/ソフトウェア>）

- ソフトウェア（英: software）は、コンピュータ分野でハードウェア（物理的な機械）と対比される用語で、**何らかの処理を行うプログラム**や、さらには関連する文書などを指す

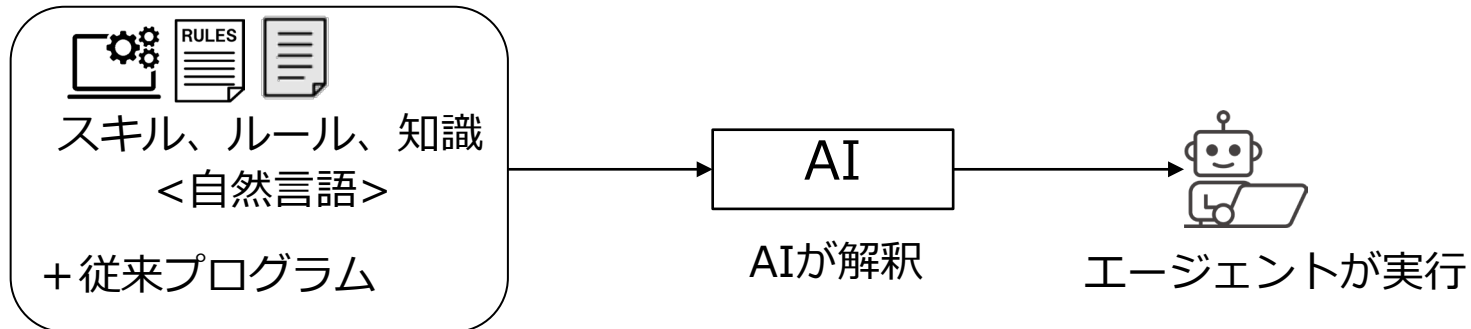
コンピュータプログラム（従来型とエージェント定義）

- コンピュータプログラム （引用：<https://ja.wikipedia.org/wiki/コンピュータプログラム>）
 - コンピュータプログラム（英語: computer program）は、コンピュータに対する命令（処理）を記述したものである。

- 伝統的なコンピュータプログラム



- AI時代のコンピュータプログラム「エージェント定義」
 - 自然言語で記述される。AIが解釈して実行できるので、本資料ではコンピュータプログラムに含める。



「従来型プログラム」と「エージェント定義」の違い

	従来型プログラム	エージェント定義
言語	プログラム言語	自然言語
実行の仕組み	コンパイル／インタプリタで命令を逐次実行	AIが意図・文脈を解釈して行動する
構成要素	アルゴリズム＋データ構造＋制御フロー	スキル・ルール・知識 ＋従来プログラム
振る舞い	決定的（同じ入力なら同じ結果）	確率的・文脈依存 （同じ指示でも結果が揺れうる）
変更の仕方	ソースを直して再ビルド	指示・知識・ルールを書き換える （再コンパイル不要）
作り手に必要なもの	プログラミング技能	ドメイン知識と、意図を言葉で伝える力
正しさの保証	テスト・型・形式検証が比較的確立	評価が難しく、幻覚・逸脱・指示漏れが起きうる
向いている仕事	仕様が固定で、再現性と正確さが最優先の処理	目標は明確だが手順が可変、判断やツール利用が必要な仕事

AI時代のソフトウェア開発

1. AI時代の開発対象は2系統（実際は組み合わせ）
 - 従来型プログラム
 - エージェント定義
 2. 従来型は、作り方を再定義する
 - AIがコードを生成する前提で、開発プロセスを再定義する
 3. **エージェント定義は、書きやすさだけで工程を省略しない**
 - エージェント定義は自然言語で行うので技術の敷居が下がるが、品質を下げないように、開発プロセスの定義が必要
 - 工程を省略しない：品質↑、生産性↓
 - 工夫しながら工程を省略する：品質→、生産性↑
- どちらにしても、AI時代の開発プロセスを検討する必要がある

←エージェント定義開発は始まったばかりで不明点が多いが、3の考えは必然と思っている

ソフトウェアに文書が含まれる。開発プロセスは、文書で具現化される。
→ソフトウェア開発において文書は重要である。
AIは文書を取り扱うことに長けているので、AI時代の開発に希望が持てる。

AI時代の開発プロセスを考える

本資料では、以下を区別する

(A) 開発対象としてのエージェント定義

(B) 開発手段としてのAIエージェント

「(B)で(A)を開発する」がこの先の主題

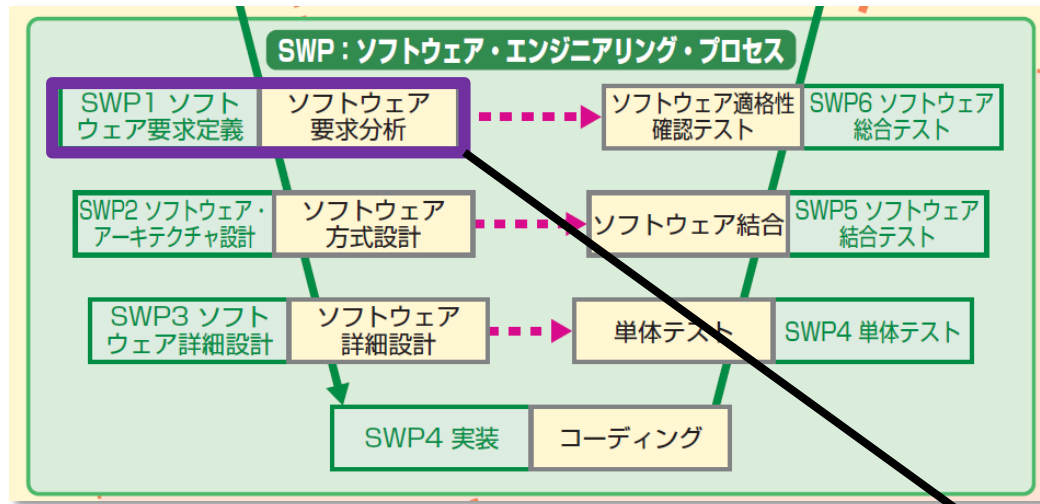
AI駆動開発

- AI駆動開発 （引用：<https://ja.wikipedia.org/wiki/AI駆動開発> ）
 - ソフトウェア開発プロセスにおいて、人工知能（AI）、特に機械学習（ML）や大規模言語モデル（LLM）を活用し、コード生成、テスト自動化、要件分析などのタスクを支援・自動化するアプローチ
 - AI駆動開発は、ソフトウェア開発の効率と品質を向上させる大きな可能性を秘めているが、その恩恵を最大限に引き出すためには、技術的・倫理的・法的な課題に継続的に取り組み、人間とAIの適切な役割分担と協働モデルを確立していく必要がある。
- つまり、AI駆動開発は、(B)を開発プロセスに組み込む作法
 - 開発手段としてのAIエージェントを利用する
- 技術進歩が急激な分野は、自分で率先して新しい取り組みをすることで、優位性を保つことができる
 - 定義は動く。だから自分でプロセスを定義する
 - 自分で取組んでいると技術を深く理解できるので、他者の提案を目利きできるようになる
 - 自分の技術力が高まり、AI時代に頼られる存在になる

さあ、自分たちで取り組んでいこう

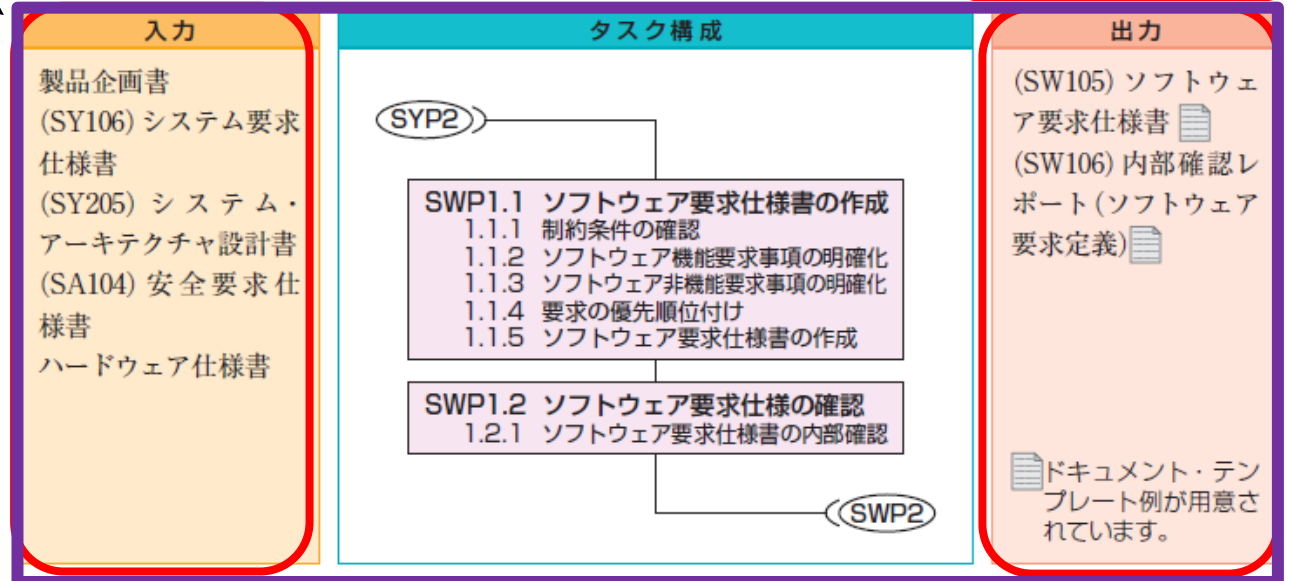
図式：開発プロセス

図引用：IPA、ESPR Ver.2.0：【改訂版】組込みソフトウェア向け開発プロセスガイド



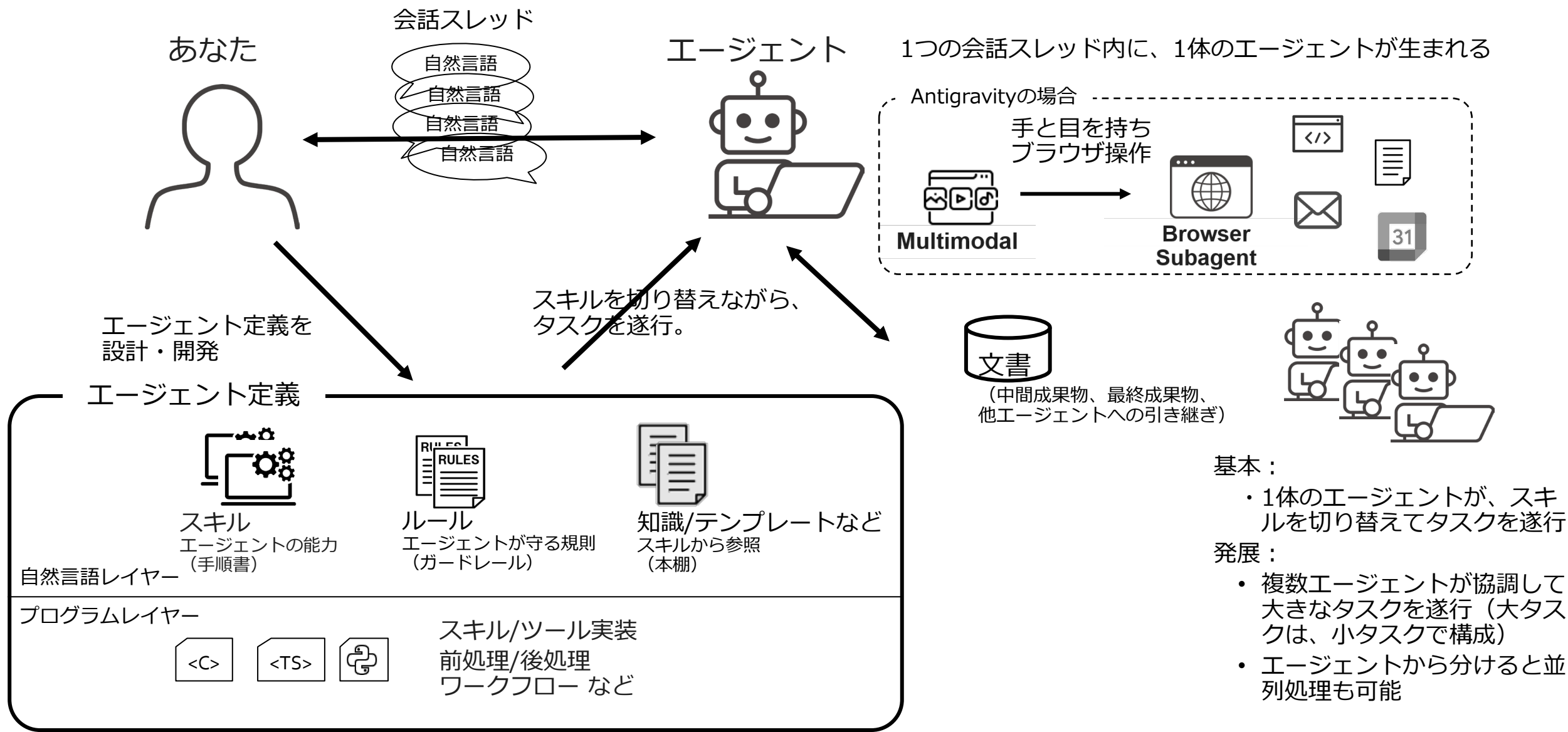
- アクティビティ毎に、求められる技術が異なる
- 各アクティビティでは、「本来の作業」と「レビュー」が行われ、そこでも求められる技術が異なる
- アクティビティ間は、**開発文書**で情報を引き渡す

開発文書



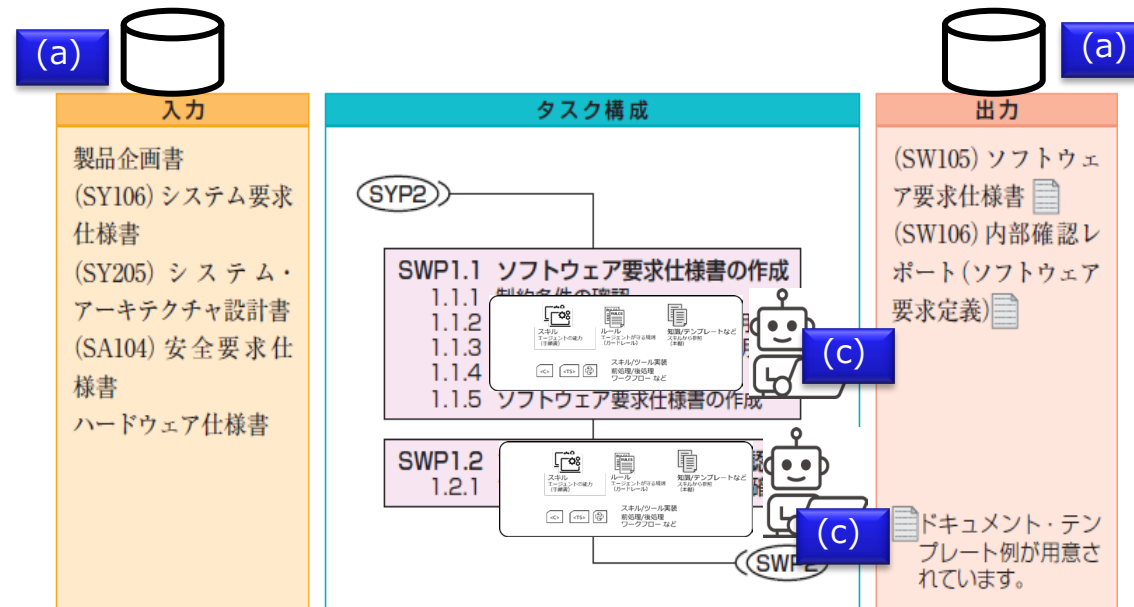
開発文書

図式：AIエージェント図解



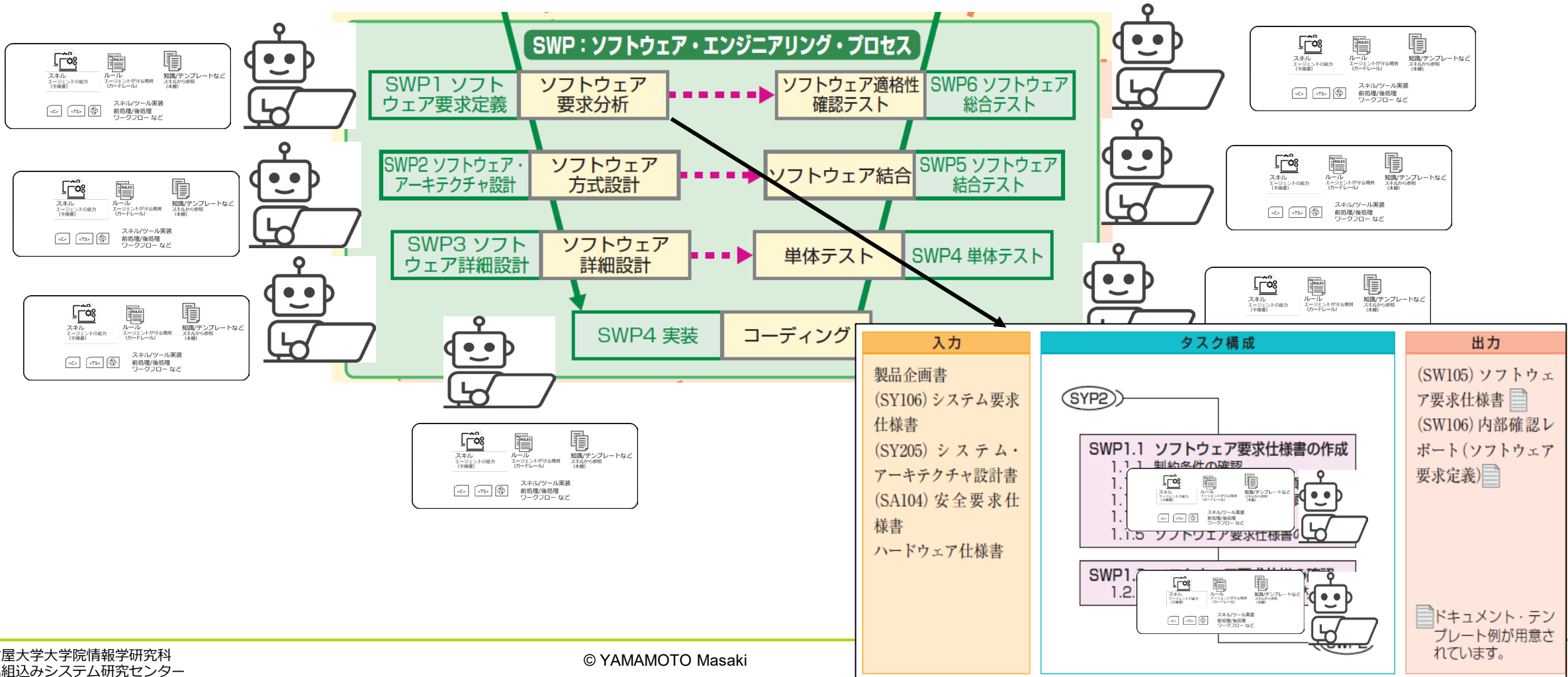
開発プロセスとAIエージェントは親和性が高い (1)

- (a) 開発プロセスは、各アクティビティで開発文書を入力し、出力する
- (b) アクティビティのタスクは「作成」と「レビュー」に分けることができ、それぞれに、専門のエージェント定義を割り当てる。
各エージェントは開発文書を読み/書きする



開発プロセスとAIエージェントは親和性が高い (2)

- 開発プロセスはアクティビティ毎に要求されるエージェント定義が異なる
 - アクティビティ毎に分ける
 - アクティビティの中に異なるタスクがある場合は、エージェント定義をタスク毎に分ける



AI時代の開発プロセス

本資料では、以下を区別する

- (A) 開発対象としてのエージェント定義
- (B) 開発手段としてのAIEージェント

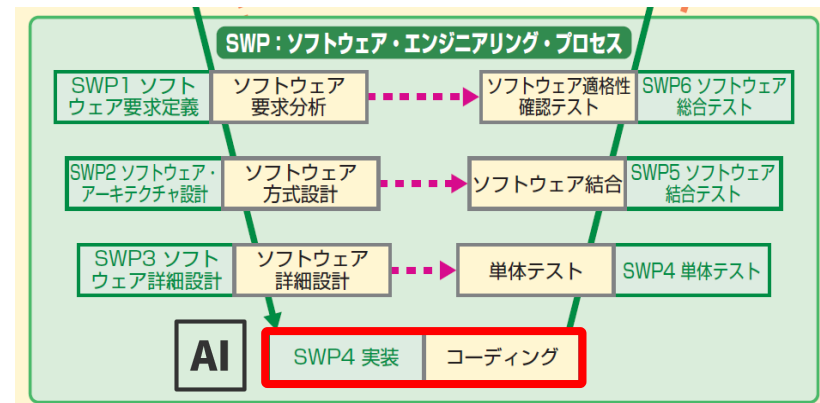
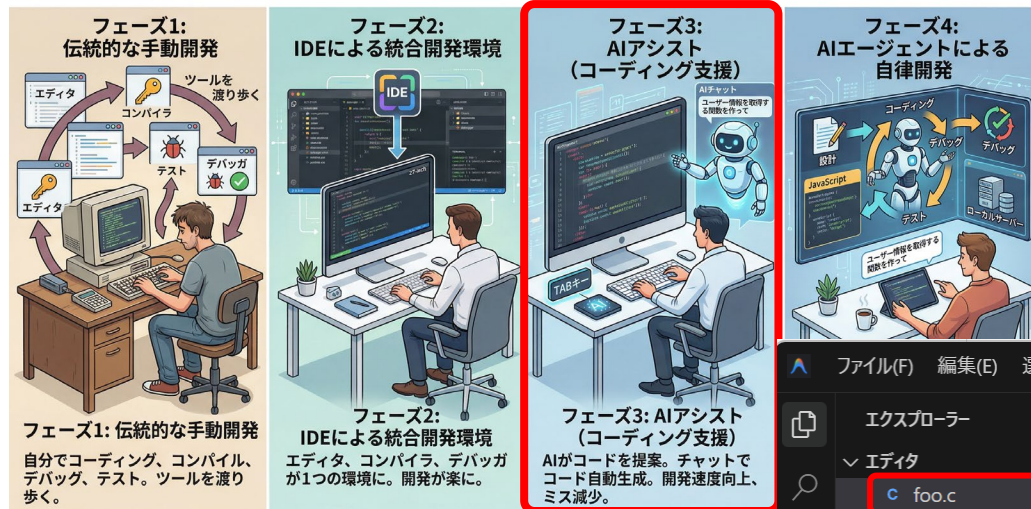
	プロセス分類	(B)の エージェ ント利用	人間 関与	従来型プ ログラム	エージェ ント定義
1	AIエディタで実装支援	なし	大	○	－
2	AIで開発文書の作成支援	なし	大	○	－
3	「詳細設計」「実装」「単体テスト」をAIEージェントに置き換える	一部	中	○	－
4	上流工程の開発文書作成/レビューをAIEージェントと協同する	中核	中	○	○
5	人間は狭義Loopの外に出てメタLoopを回す	中核	小	○	○

- ・ エージェント定義の開発は、AIEエージェント登場後に必要になった
- ・ なるべくAIEエージェントの恩恵を受けながら、エージェント定義の開発を行うこととしたので、**エージェント定義の開発は、プロセス分類4と5のみとした**

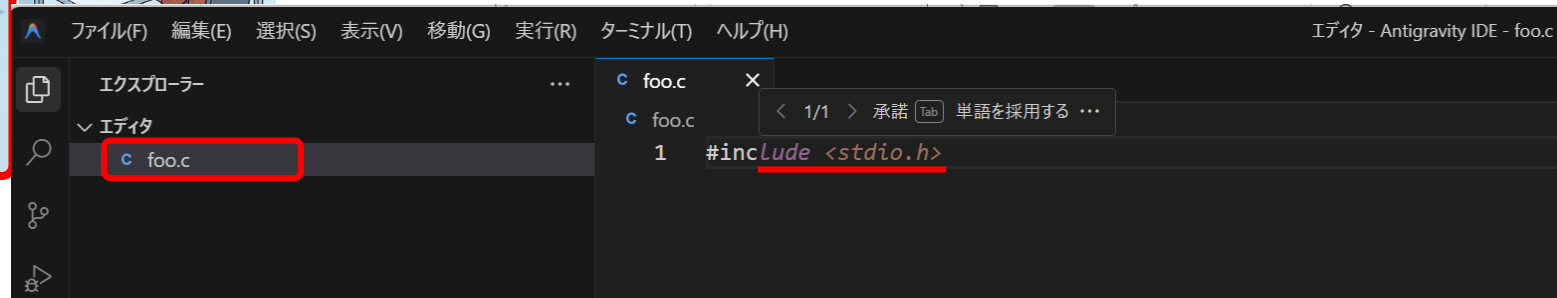
1. AIEDィタで実装支援

コードを打ち始めると、AIが続くコードを提案する。
AI提案を承認(Tabキー押下)すると、コードが補完

ソフトウェア開発の進化：手動から自律へ



例：Antigravity IDEで foo.c ファイルを作成し、#inc とタイプするとAIが続くコードを提案（AIはCソースの編集であると認識している）



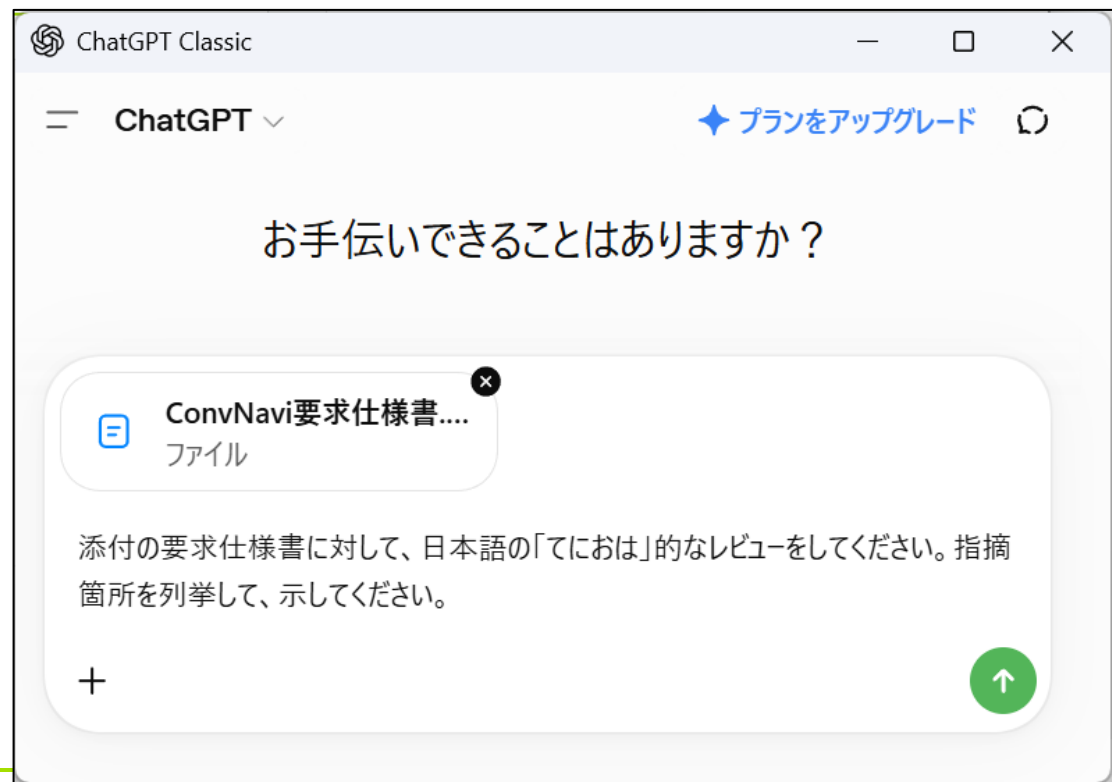
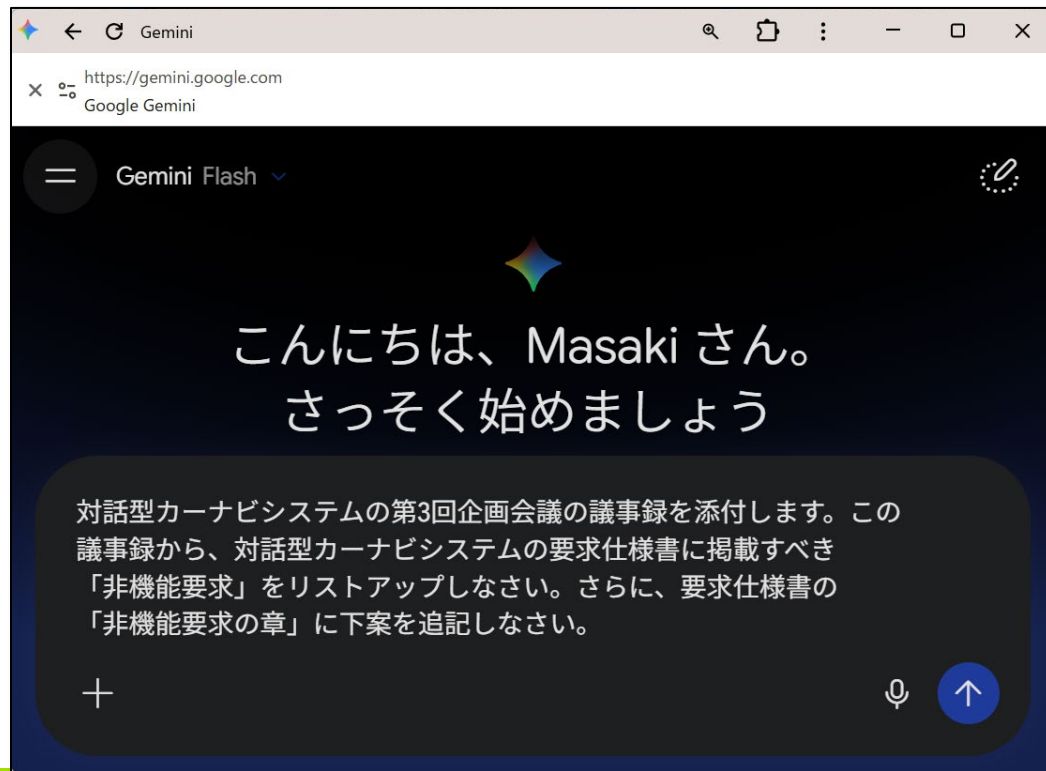
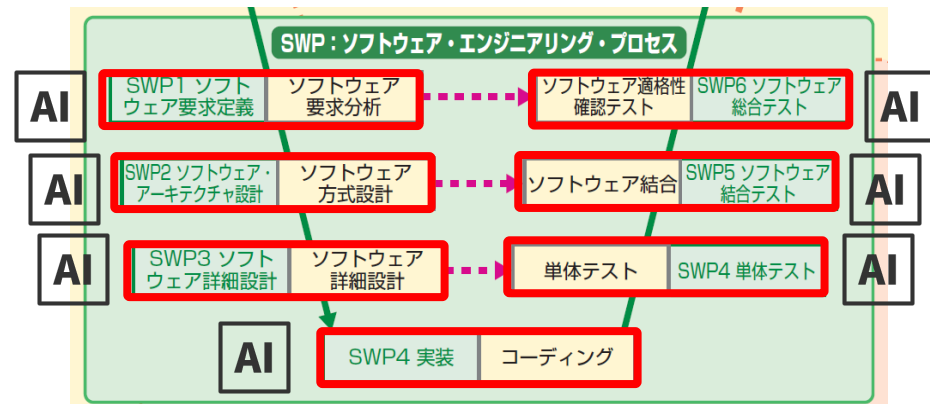
AIEDィタを用いることで、コーディングの生産性と品質が向上。

- 次に編集すべき箇所へカーソルを自動ジャンプ
- 複数ファイルを横断した一括編集
- 既存コードの解説・要約 など

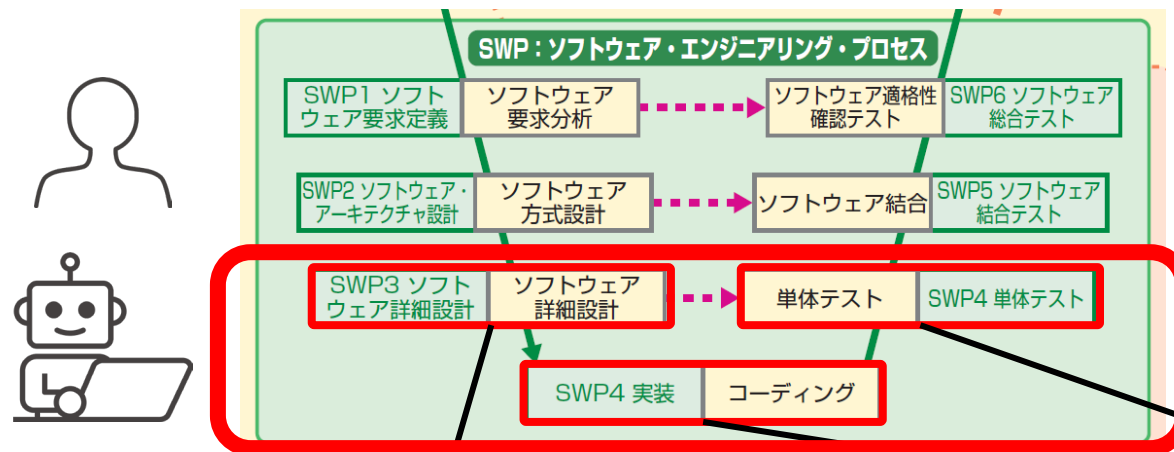
2. AIで開発文書の作成支援

各アクティビティの一部タスクをAIに手伝わせる

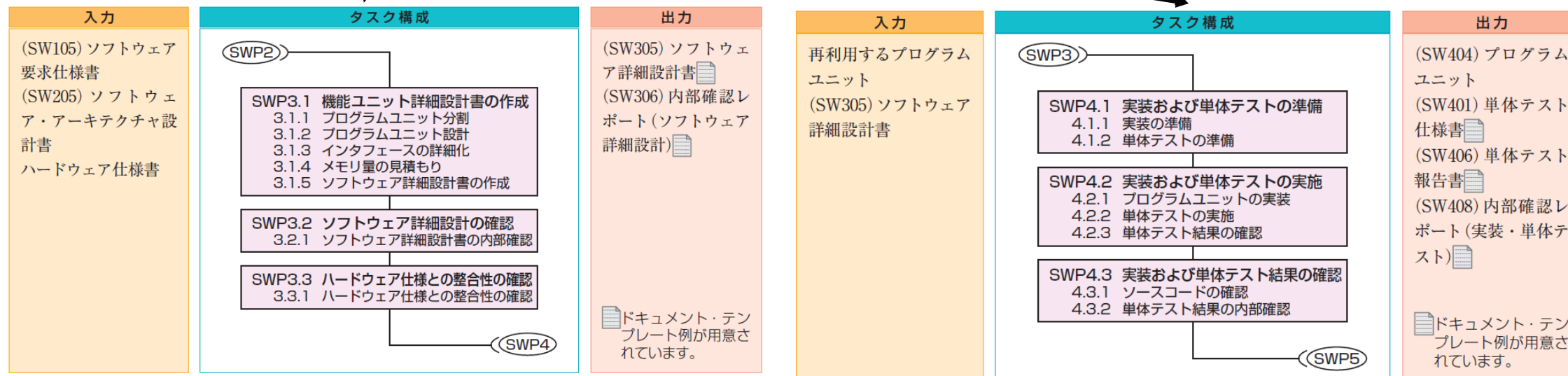
- 開発文書の作成
- 開発文書のレビュー



3. 「詳細設計」「実装」「単体テスト」をAIエージェントに置き換える

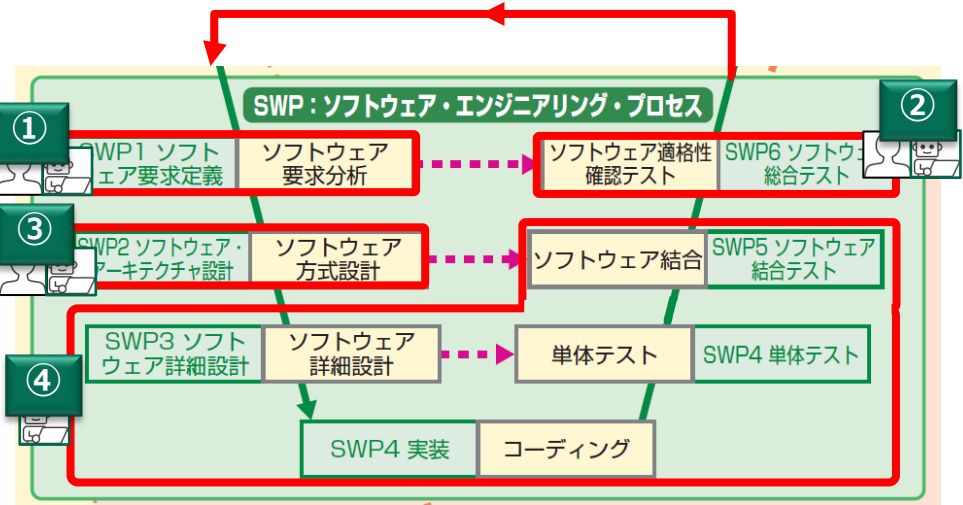


2025年末頃から、LLMのコーディング能力が高くなり、AIにコーディングを任せることが可能な水準まで到達。エージェント技術も進化して、バグがとれるまでコーディングを行うエージェントが、実用水準まで到達。

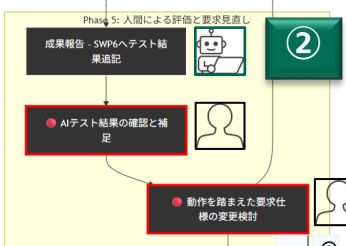
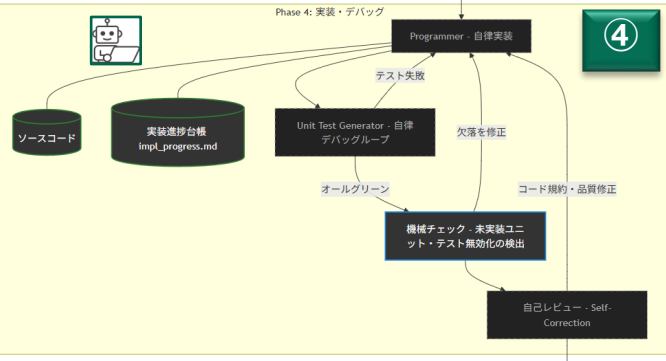
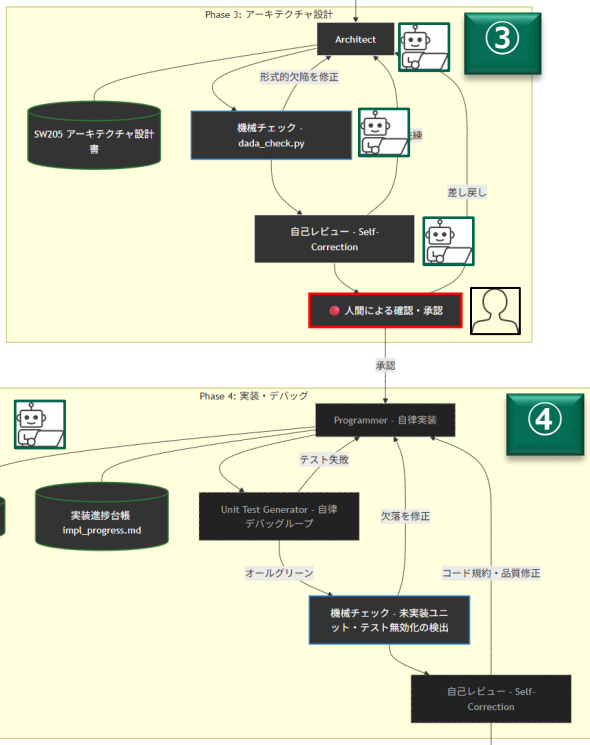
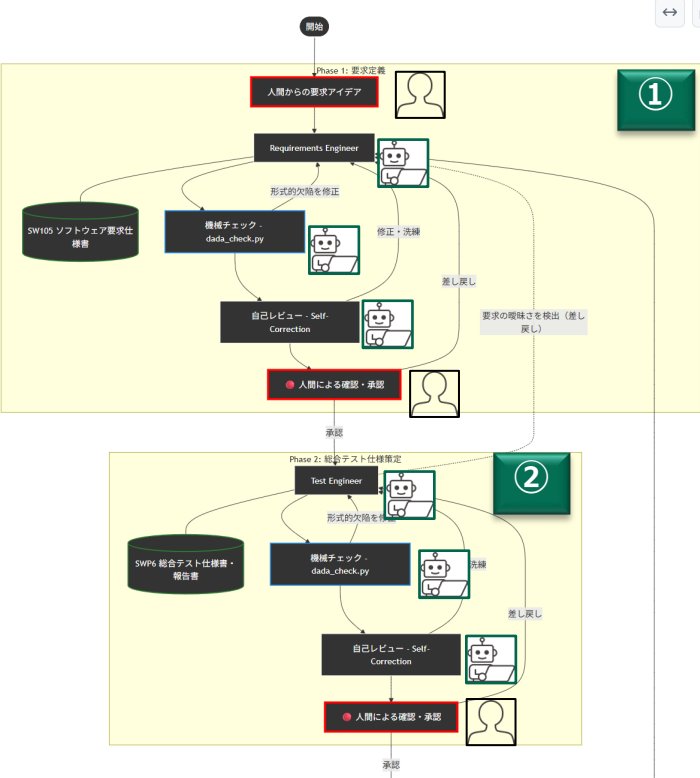


4-1. 上流工程の開発文書作成/レビューをAIEージェントと協同する

例としてのDADA (Document-and-Agent-Driven Agile) 開発プロセス (後ほど詳説)



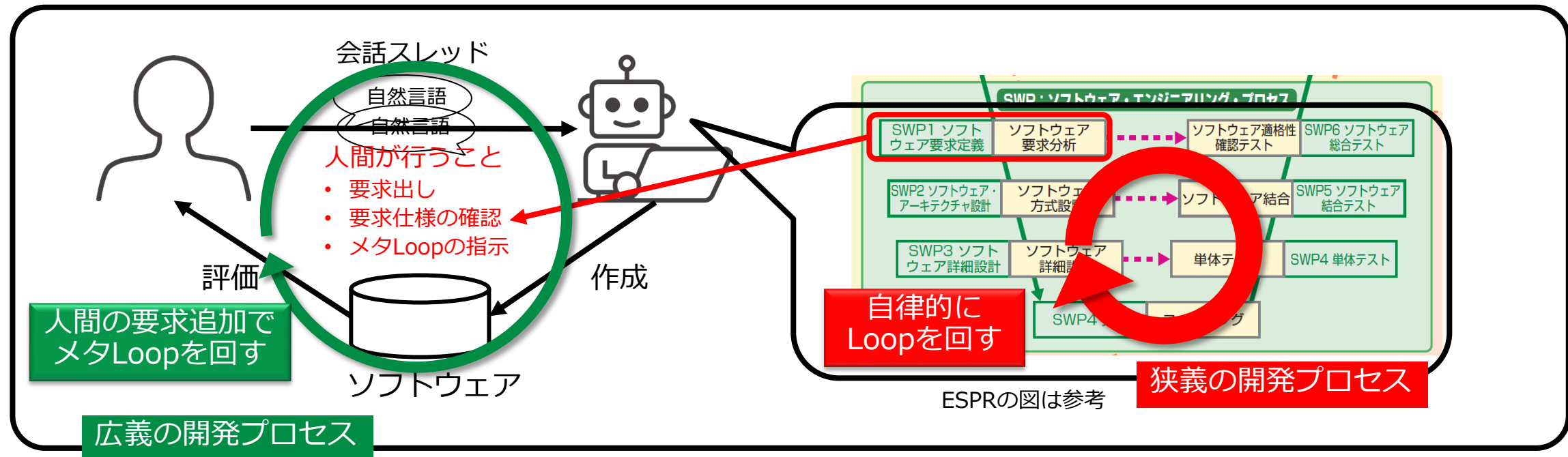
- AIエージェントの自律作業を増やす
 - 要求定義とそのレビュー
 - アーキテクチャ設計とそのレビュー
 - 実装とデバッグとユニットテスト
 - 総合テストとそのレビュー
- 開発文書が人とエージェントのインタフェース
 - 要求仕様書
 - アーキテクチャ設計書
 - 総合テスト仕様書・報告書



DADAはAIと協同してESPRに従い開発する一例に過ぎない
各自、自社の開発プロセスやA-SPICEなどへのAI組み込みを考案しよう

5-1.人間は狭義Loopの外に出てメタLoopを回す

- エージェントは自律開発をする（狭義のLoop）
- 人間はメタLoopを回す
 - メタLoop：成果を見て要求を足し、狭義Loopを再起動する外側のループ



狭義Loopを人で止めない。品質は評価セットとメタLoopで見る。
極めて高度な品質（再利用性、効率性など）を要求しないプログラムの開発に最適。

4-2.上流工程の開発文書作成/レビューをAIエージェントと協同する

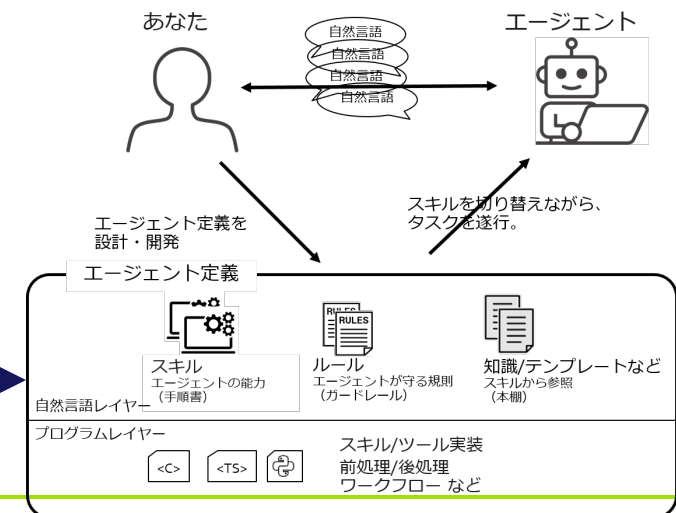
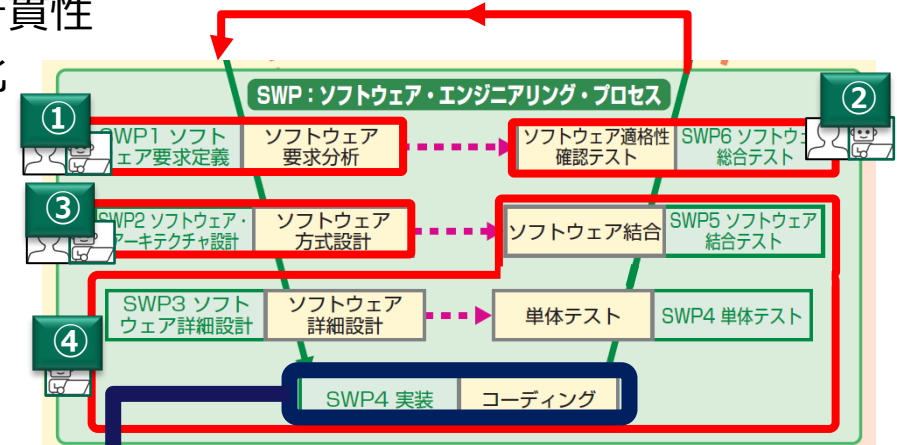
- 従来型プログラム開発で「4. 上流工程の開発文書作成/レビューをAIエージェントと協同する」を提案
- 開発プロセスに以下の工夫を加える
 - 最終成果物は「従来型プログラム」ではなく「エージェント定義」
 - レビュー対象がコードではなく、スキル・ルール・知識の一貫性
 - テストが決定的でないので、評価セットと許容範囲を文書化
 - 要求を工夫する（下記）
- DADAプロセスは最終成果物を限定しない。要求に例えば以下を加えると「エージェント定義」の開発可能

エージェント定義の要求仕様の核

最終成果物は「エージェント定義」一式です。つまり、次の2種類です。私が出す要求を満たすようにエージェント定義を作ってください。その際に、(1)と(2)の特徴を活かして、適切に「エージェント定義」を作成してください。

(1)自然言語で記述された「スキル」「ルール」「知識/テンプレートなど」。文脈依存の知的な処理に向く。

(2)従来型のプログラム言語、例えばタイプスクリプトやC言語など、で記述されたソースファイル。機械的な処理に向く。



5-2.人間は狭義Loopの外に出てメタLoopを回す

- 「5-1.人間は狭義Loopの外に出てメタLoopを回す」は従来型プログラムを開発したが、同様の方法でエージェント定義を開発する
- 自然言語で記述されるエージェント定義部分はLLMで確率的に動作するので、以下を注意
 - エージェントとのチャットを明確に行い曖昧さを排除する→エージェント定義の開発が安定する
 - 予めLoop内の評価セットを強化する

